Raport laborator TWA

Date generale

Student: Zlatovcen Bogdan
Profesor: Ovidiu Gherman
Tema: Contact Manager

1. Scopul lucrării

Scopul lucrării a fost realizarea unei aplicații web practice pentru gestionarea contactelor personale, utilizând tehnologii moderne pentru dezvoltare web. Aplicația demonstrează folosirea unui backend FastAPI, a unui frontend React cu Ant Design, a unei baze de date SQLite3, a autentificării prin JWT, precum și containerizarea cu Docker și pregătirea deployment-ului pe Render.com.

Prin această lucrare au fost aplicate concepte importante precum proiectarea unui API REST, validarea datelor, securizarea endpoint-urilor, separarea responsabilităților în cod, gestionarea stării în interfața React și configurarea aplicației prin variabile de mediu.

2. Descrierea aplicației

Contact Manager este o aplicație web în care utilizatorii pot crea un cont, se pot autentifica și pot administra propria listă de contacte. După autentificare, fiecare utilizator poate adăuga contacte noi, poate vizualiza contactele existente, poate edita informațiile unui contact și poate șterge contactele care nu mai sunt necesare.

Datele fiecărui utilizator sunt izolate. Un utilizator autentificat poate accesa doar contactele asociate contului său, iar cererile către endpoint-urile protejate necesită un token JWT valid.

3. Tehnologii utilizate

- FastAPI a fost utilizat pentru backend și pentru expunerea endpoint-urilor REST.
- React.js a fost utilizat pentru construirea interfeței frontend.
- Ant Design a fost utilizat pentru componente vizuale precum formulare, tabele, carduri, butoane și ferestre modale.
- SQLite3 a fost utilizat ca bază de date serverless, accesată prin modulul standard Python `sqlite3`.
- JWT a fost utilizat pentru autentificare și autorizarea cererilor către endpoint-urile protejate.

- Docker a fost utilizat pentru containerizarea aplicației și rularea ei într-un mediu reproductibil.
- Render.com a fost utilizat ca platformă țintă pentru deployment.
- GitHub a fost utilizat pentru găzduirea codului sursă.

4. Arhitectura aplicației

Aplicația este structurată în trei componente principale: interfața React, API-ul FastAPI și baza de date SQLite3. Fluxul principal este următorul:

Browser React UI -> FastAPI API -> SQLite3 database

Frontend-ul trimite cereri HTTP către endpoint-urile backend-ului. Endpoint-urile publice permit înregistrarea și autentificarea, iar endpoint-urile pentru contacte sunt protejate. După autentificare, clientul primește un token JWT și îl trimite la cererile următoare în header-ul `Authorization: Bearer <token>`.

În producție, aplicația React este construită cu Vite, iar fișierele statice rezultate sunt servite de FastAPI din același serviciu Docker. Endpoint-urile API sunt prefixate cu `/api`, pentru a nu intra în conflict cu rutele frontend-ului.

5. Baza de date SQLite3

Baza de date este implementată cu SQLite3 și este inițializată automat la pornirea aplicației prin mecanismul lifespan din FastAPI. Conexiunea folosește `sqlite3.Row` pentru acces clar la coloane și activează `PRAGMA foreign_keys = ON` pentru respectarea relațiilor dintre tabele.

Tabelul `users` conține datele utilizatorilor: identificatorul, username-ul, hash-ul parolei și data creării contului. Tabelul `contacts` conține contactele: identificatorul, `user_id`, numele, telefonul, email-ul, notele și datele de creare și actualizare.

Relația dintre tabele este realizată prin cheia străină `contacts.user_id`, care referă `users.id`. Astfel, fiecare contact aparține unui singur utilizator. Toate interogările pentru contacte filtrează după `user_id`, ceea ce asigură faptul că fiecare utilizator vede și modifică doar contactele proprii.

6. Autentificare și securitate

Înregistrarea se face prin trimiterea unui username și a unei parole. Username-ul este normalizat prin eliminarea spațiilor și transformarea în litere mici. Parola nu este salvată niciodată în text clar, ci este hash-uită folosind bcrypt prin biblioteca passlib.

La autentificare, aplicația verifică parola introdusă cu hash-ul salvat în baza de date. Dacă datele sunt corecte, backend-ul generează un token JWT cu durată

limitată. Frontend-ul salvează token-ul în `localStorage` și îl trimite la cererile protejate în formatul `Authorization: Bearer <token>`.

Endpoint-urile protejate validează token-ul și identifică utilizatorul curent. Dacă token-ul lipsește, este invalid sau a expirat, backend-ul returnează codul HTTP 401.

7. Endpoint-uri API

Method	Endpoint	Description
GET	`/healthz`	Verifică dacă aplicația rulează
POST	`/api/auth/register`	Creează un cont nou
POST	`/api/auth/login`	Autentifică utilizatorul și returnează token JWT
GET	`/api/auth/me`	Returnează utilizatorul autentificat
GET	`/api/contacts`	Returnează contactele utilizatorului curent
GET	`/api/contacts/{contact_id}`	Returnează un contact dacă aparține utilizatorului curent
POST	`/api/contacts`	Creează un contact nou
PUT	`/api/contacts/{contact_id}`	Actualizează un contact existent
DELETE	`/api/contacts/{contact_id}`	Șterge un contact existent

8. Interfața React cu Ant Design

Interfața este construită în React.js și utilizează componente Ant Design pentru un aspect curat și ușor de folosit. Aplicația are un ecran de autentificare unde utilizatorul poate alege între login și register. Erorile și mesajele de succes sunt afișate cu sistemul `message` din Ant Design.

După autentificare, utilizatorul ajunge la ecranul principal cu lista de contacte. Acesta include un header cu numele aplicației, username-ul utilizatorului curent și butonul de logout. Lista de contacte este afișată într-un tabel și poate fi filtrată local după nume, telefon sau email.

Adăugarea și editarea contactelor sunt realizate printr-o fereastră modală Ant Design. Formularul conține câmpurile nume, telefon, email și note. Pentru ștergere este folosit un `Popconfirm`, astfel încât utilizatorul să confirme acțiunea înainte ca datele să fie eliminate.

Componente Ant Design utilizate: `Layout`, `Header`, `Card`, `Form`, `Input`, `Button`, `Table`, `Modal`, `Space`, `Typography`, `message`, `Popconfirm` și `Empty`.

9. Flux frontend

Pasul 1 - Ecran autentificare (login/register)

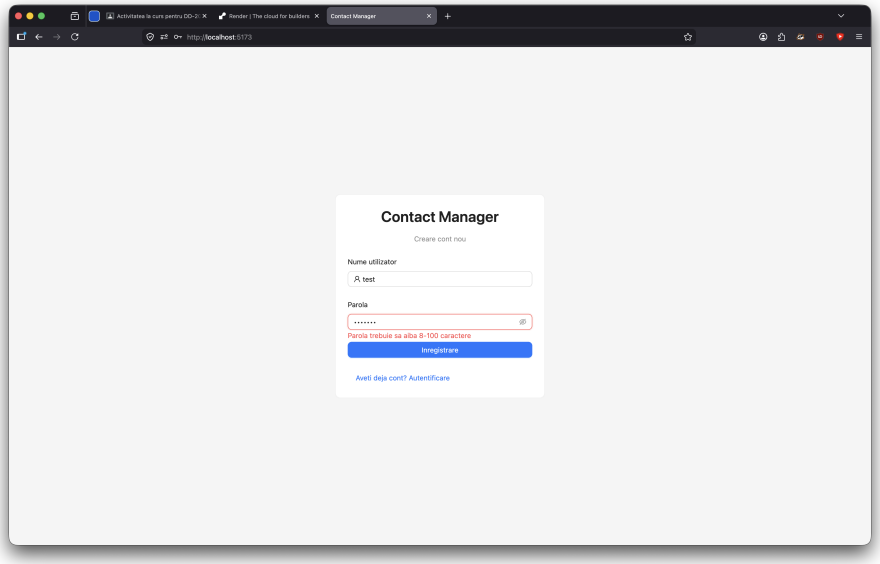

Figure 1: Ecran autentificare

Utilizatorul deschide aplicația și are acces la formularul de autentificare sau înregistrare.

Pasul 2 - Înregistrare reușită

După completarea datelor, aplicația confirmă crearea contului, iar utilizatorul poate continua cu autentificarea.

Pasul 3 - Lista de contacte

După login, utilizatorul este redirecționat în ecranul principal unde poate vizualiza și căuta contactele.

Pasul 4 - Adăugare contact nou

Prin butonul de adăugare se deschide formularul din modal. Utilizatorul completează datele, salvează, iar contactul apare în listă.

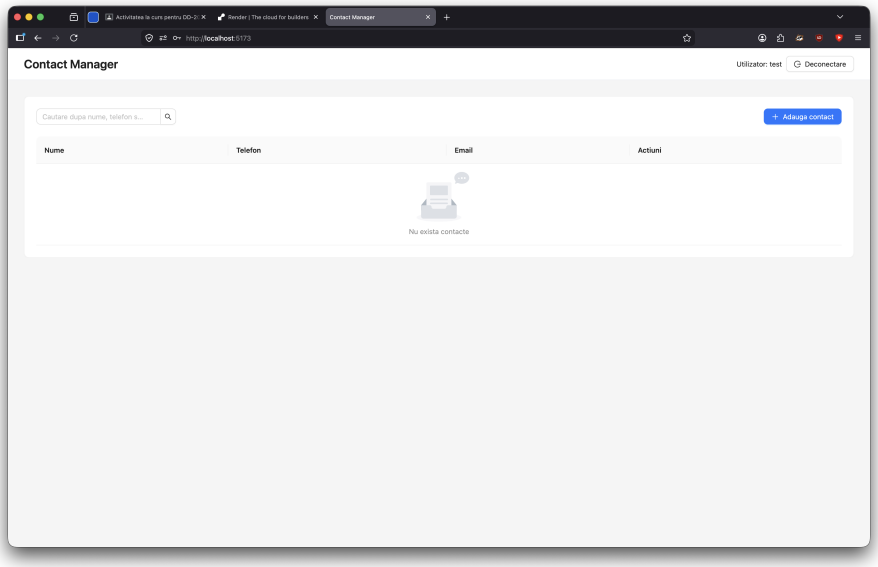

Figure 2: Înregistrare reușită

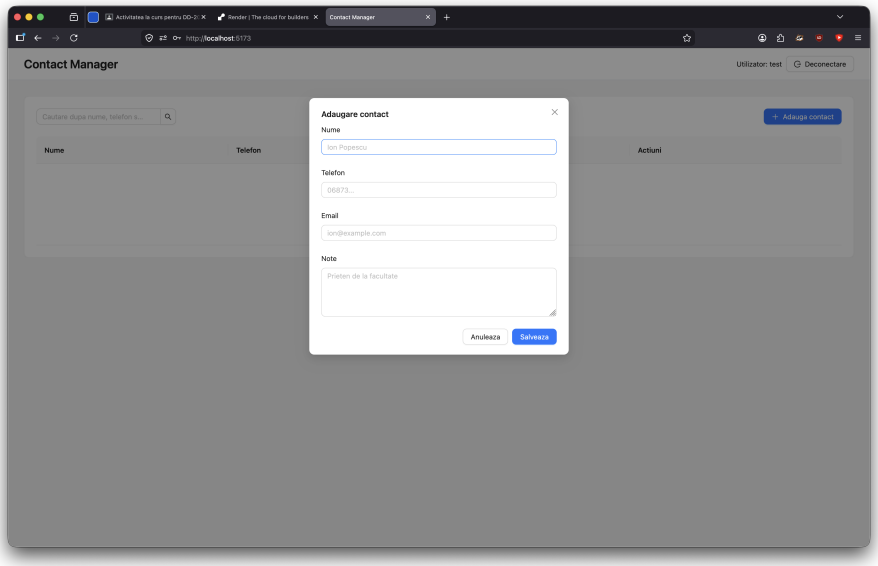

Figure 3: Lista contacte

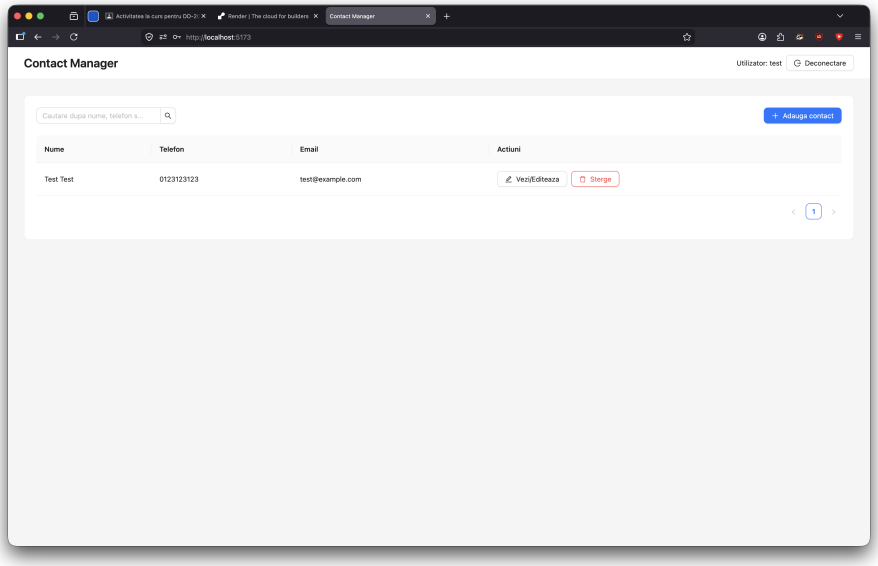

Figure 4: Adăugare contact

10. Containerizare cu Docker

Docker este folosit pentru a rula aplicația într-un mediu uniform, indiferent de sistemul pe care este executată. Containerizarea simplifică deployment-ul și reduce diferențele dintre mediul local și mediul de producție.

Fișierul Dockerfile este multi-stage. Prima etapă folosește Node.js pentru instalarea dependențelor frontend și construirea aplicației React. A doua etapă folosește Python 3.12 slim, instalează dependențele backend și copiază fișierele statice generate de React în aplicația FastAPI.

La rulare, FastAPI pornește cu uvicorn și servește atât API-ul, cât și frontend-ul construit.

11. Deployment pe Render.com

Aplicația a fost deployată pe Render.com folosind serviciu Docker, iar aplicația rulează într-un singur serviciu care servește atât API-ul FastAPI, cât și frontend-ul React build-uit.

12. Concluzii

În cadrul acestei lucrări a fost realizată o aplicație web completă, cu frontend, backend, bază de date, autentificare și deployment pregătit. Proiectul a permis aprofundarea conceptelor de API REST cu FastAPI, lucrul cu SQLite3, autentificarea cu JWT, securizarea parolelor prin bcrypt, dezvoltarea interfețelor cu React și Ant Design, precum și containerizarea cu Docker.

Lucrarea demonstrează modul în care o aplicație web simplă, dar completă, poate fi construită și pregătită pentru rulare locală și pentru deployment pe o platformă cloud precum Render.com.